ASSON UI --- V1 Preservation, V2 Implementation & Repository Governance Directive

Document type: Engineering implementation directive\ **Project:** ASSON UI Student Platform --- University of Ibadan Chapter\ **Repository:** `https://github.com/emmanuelikeh2993/asson_website`\ **Primary branch:** `main`\ **Backend:** Existing Supabase project\ **Existing hosting:** Both Netlify and Vercel\ **Status:** Continue the existing V1 implementation; do not initialise a new project\ **Purpose:** Give Antigravity or another engineering AI one authoritative set of instructions for auditing, evolving, documenting and maintaining the existing ASSON UI platform.

---

0. Read this first: non-negotiable operating instructions

This is an existing application. Version 1 has already been developed, connected to Supabase, populated with academic content, deployed and made available through **both Netlify and Vercel**.

You are not being asked to create a new website, scaffold a replacement application, repeat project onboarding, or redesign the entire product from scratch.

Your job is to continue the existing implementation carefully.

0.1 Preserve the existing project

1 Work from the existing GitHub repository and its current `main`

branch unless the project owner explicitly directs otherwise.

1 Inspect the repository, current Git history, environment

configuration, Supabase integration and both hosting deployments before making changes.

1 Preserve the existing frontend framework, component conventions,

routing, design system, database schema, storage setup and working user flows unless a documented reason justifies changing them.

1 Do not replace working components with newly generated alternatives

simply because you prefer another implementation.

1 Do not create a second application, a second Supabase project, a

duplicate database, or a replacement repository.

1 Do not assume that the handover document accurately describes the

current code. Verify claims against the repository, live deployments and Supabase configuration where authorised.

1 Do not make destructive changes to production data, database schema,

storage files, DNS, domains, hosting configuration or deployment integrations without explicit approval.

1 Do not deploy to production, change a production domain, or

disconnect either hosting provider without explicit approval.

0.2 Both hosting deployments must be preserved

The application is currently hosted on **Netlify and Vercel**. Treat both as existing deployments that may serve real users.

During the audit:

- Identify the production URL and deployment status for each provider.
- Identify which GitHub repository and branch each provider is

connected to.

- Inspect build commands, output directories, environment-variable

names and relevant configuration files.

- Determine whether both deployments are actively maintained, whether

they serve the same code, and whether their environment configurations are consistent.

- Identify any differences in redirects, SPA fallback behaviour,

headers, caching, domain configuration or build settings.

- Record the findings and any risks.

Do not decide unilaterally that one provider is redundant. Do not migrate, disable, disconnect or delete either deployment. Recommend a primary deployment arrangement only after documenting the consequences and obtaining the owner's approval.

0.3 Work in controlled stages

Use this sequence:

1. Read and understand this directive.
2. Inspect the current repository and deployments without changing

anything.

1. Produce the audit and gap analysis.
2. Present critical findings and a proposed implementation order.
3. Wait for approval before making changes.
4. Implement only the approved phase.
5. Test the affected workflows and security boundaries.
6. Report changed files, migrations, tests, deployment implications and

unresolved risks.

1. Wait for approval before production deployment.

If a task cannot be safely completed because information or permission is missing, stop and explain what is needed. Do not guess.

---

1. Product vision

ASSON UI is the digital academic home of the **Association of Statistics Students of Nigeria (ASSON), University of Ibadan Chapter**.

Its purpose is to make reliable academic information and student support easy to find, while helping successive association administrations maintain a useful institutional platform.

The platform should evolve from a public academic resource library into a dependable student-facing academic hub. It should not become a collection of disconnected features.

1.1 Product principles

- **Free access to core academic resources:** Students must be able to

browse and download public academic materials without compulsory login.

- **Mobile-first:** Optimise for Android and iPhone users, especially

students using limited data and smaller screens.

- **Reliable information:** Academic materials, notices and

opportunities should have clear metadata, sources and verification information.

- **Privacy by design:** Private feedback and welfare submissions must not be exposed to the public or unauthorised executives.
- **Security enforced by the backend:** A frontend lock, hidden route or PIN screen is not a substitute for server-side authorisation.
- **Institutional continuity:** The association must retain control of its accounts, data, domains and documentation across administrations.
- **Maintainability:** Prefer simple, documented features that the next administration can operate over technically impressive features that nobody can maintain.
- **No fabricated content:** Never invent course materials, official notices, contacts, scholarship details, statistics, endorsements or verification claims.
- **Graceful operation on free tiers:** Design with actual provider quotas and limits in mind. Do not introduce paid services or subscriptions without approval.

1.2 What success means

The platform should help a student answer these questions quickly:

1. Where can I find the academic material for my course?
2. Where can I find relevant past questions and, where available, verified solutions?
1. What official notice, deadline or opportunity should I know about?
2. Where can I get help with an academic or student issue?
3. How can I report a broken link, incorrect information or missing resource?
1. Can I trust the information displayed?
2. Will the platform still work when the current executives leave office?

Do not use raw page views or feature count as the only measures of success. Measure useful actions and reliability.

---

2. Existing V1 baseline --- verify, do not assume

The previous engineering handover describes the following implementation. Treat these as **reported facts requiring code/configuration verification**, not as guaranteed current behaviour.

2.1 Reported project foundation

- GitHub repository: `emmanuelikeh2993/asson_website`
- Main branch: `main`
- Existing Supabase cloud project and connected frontend.
- Existing Netlify deployment.
- Existing Vercel deployment.
- Mobile-first frontend with responsive navigation and touch-friendly cards.
- Public browsing and downloads without compulsory student accounts.
- Existing admin area and moderation workflow.
- Existing notices and opportunities pages.

- Existing analytics interface.
- Existing academic resource and course catalogue.

2.2 Reported database tables

The handover document reports these 13 tables:

- `courses`
- `resources`
- `notices`
- `opportunities`
- `faqs`
- `profiles`
- `user_roles`
- `submissions`
- `bookmarks`
- `audit_events`
- `content_reports`
- `site_settings`
- `executive_contacts`

Verify the actual schema, foreign keys, constraints, indexes, triggers, functions, policies and data. Do not assume the table list is complete or that each table is used correctly.

2.3 Reported content

The handover document reports:

- 625 catalogued academic documents.
- 57 departmental and faculty courses.
- Materials spanning 100L to 400L and postgraduate levels.
- Materials covering examination years from 2018 to 2025.
- Notices containing academic calendar dates, examination schedules

and association announcements.

- An opportunities page containing scholarship and internship

listings.

Verify actual counts, completeness, duplicates, broken file links, metadata quality, resource status and the meaning of the word "verified". Do not claim that every document is verified unless a defined review process supports that claim.

2.4 Reported student workflows

- Public resource browsing and downloads.
- Anonymous resource contributions.
- Submission reference codes.
- A local Student Locker / dashboard using browser storage.
- Public notices and opportunities.
- Mobile navigation and quick course filters.

Test these workflows in the actual application. Identify any interface that is only visual, incomplete, broken or backed by placeholder data.

2.5 Reported admin and analytics workflows

The handover describes an admin console with:

- An executive passkey/PIN gate.
- Resource moderation and revision feedback.
- Download and visit analytics.
- Course-demand ranking.
- Date-range selection.

Inspect the actual implementation. In particular, establish whether every administrative operation is independently authorised by Supabase or another trusted backend. A client-side gate alone is not acceptable protection for privileged database actions.

---

3. Immediate audit: read-only V1 reality check

This is the first task. Do not implement new features during this stage.

Inspect the codebase and the authorised project configuration. Do not alter source files, database records, storage contents, policies, environment variables, DNS or hosting settings during the audit.

3.1 Repository and application architecture

Document:

- Framework and version.
- Language and build tooling.
- Package manager and dependency versions.
- Main source directories and their responsibilities.
- Route definitions and page components.
- Shared layout, navigation and design-system components.
- State management and data-fetching patterns.
- Supabase client initialisation and data-access conventions.
- Form validation and error-handling patterns.
- Existing tests and scripts.
- Build and deployment configuration.
- Existing documentation and project conventions.

Do not introduce a new framework or rewrite the app as part of the audit.

3.2 Page and feature inventory

For every route/page, record:

- Route and page name.
- Purpose.
- Whether it is public or restricted.
- Main components.
- Data source.
- Main user actions.
- Whether those actions work.
- Mobile behaviour.

- Accessibility concerns.
- Empty, loading and error states.
- Missing or placeholder content.

Classify each feature as:

- **Working:** end-to-end behaviour has been tested.
- **Partially working:** some parts work, but important behaviour is missing.
- **Visual only:** interface exists but does not perform the advertised action.
- **Broken:** the expected workflow fails.
- **Not implemented:** no meaningful implementation exists.
- **Unverified:** insufficient evidence to classify it.

3.3 Supabase database and security audit

Inspect the actual Supabase schema and policies, using authorised read-only access wherever possible.

For every table, record:

- Purpose.
- Columns and data types.
- Primary and foreign keys.
- Constraints and indexes.
- Triggers and functions.
- RLS enabled status.
- SELECT, INSERT, UPDATE and DELETE policies.
- Roles permitted by each policy.
- Whether the policy checks ownership or administrative authority.
- Whether private information can be accessed by anonymous users.
- Whether users can modify fields they should not control.

Test access boundaries where safe and authorised. At minimum, reason about:

1. Anonymous public visitor.
2. Ordinary authenticated student, if student authentication exists.
3. Verified contributor, if such a role exists.
4. Executive or content moderator.
5. Highest-level administrator.

Do not assume that enabling RLS makes a table secure. Check the actual policies and the way the frontend calls the database.

3.4 Storage and upload audit

The earlier handover describes a public `resources` bucket and public INSERT permission. Verify the current configuration.

Inspect:

- Bucket visibility.
- File-size limits.
- Allowed MIME types and extensions.

- Object naming and path strategy.
- INSERT, SELECT, UPDATE and DELETE policies.
- Whether anonymous users can upload directly.
- Whether uploaded objects can overwrite existing objects.
- Whether uploads are validated before becoming publicly accessible.
- Whether moderation status is enforced before a resource appears in

the public library.

- Whether untrusted files can cause storage abuse or unsafe downloads.

A public academic download bucket may be appropriate. Unrestricted public upload permissions are not automatically appropriate.

Recommend a safe anonymous contribution architecture that preserves low friction. Do not implement a replacement until approved.

3.5 Admin authentication and authorisation audit

The handover describes a frontend executive PIN gate and includes a PIN value. Treat any PIN or credential disclosed in a tracked document, prompt, screenshot or repository as compromised.

- Do not reproduce any exposed PIN in this directive, source code,

documentation or reports.

- Do not use a hardcoded shared PIN as the sole protection for

privileged actions.

- Establish whether actual authentication exists.
- Establish how executive roles are assigned and verified.
- Establish whether the backend independently enforces each privileged

action.

- Review session handling, credential storage, logout and account

recovery.

- Check whether a user can bypass the UI and call privileged database

operations directly.

- Recommend immediate credential rotation if an exposed value is still

active.

Do not change credentials or production access without explicit authorisation.

3.6 Analytics audit

For each metric, document its exact collection method and limitations:

- Page views.
- Unique visits.
- Downloads.
- Course views.
- Course demand.
- Submission counts.
- Moderation activity.

Distinguish browser-local records from server-side events. Explain how duplicate visits, bots, shared devices, blocked scripts and repeat downloads affect the figures.

Do not label a metric "unique students" unless the system has a defensible way to identify unique students. Prefer privacy-preserving aggregate analytics and avoid unnecessary collection of personally identifiable information.

3.7 Hosting audit --- Netlify and Vercel

Both providers are in use and must be included in the audit.

For each provider, document:

- Project/site name.
- Production URL and connected custom domains.
- Connected GitHub repository and branch.
- Automatic deployment behaviour.
- Build command.
- Output directory.
- Environment-variable names (never print secret values).
- Redirect and rewrite configuration.
- SPA fallback behaviour.
- Security headers and caching configuration.
- Current production deployment status.
- Whether the deployed commit matches GitHub `main`.
- Any provider-specific differences or failures.

Do not print secrets in the audit. Do not copy secret values into GitHub, Markdown, screenshots or chat output.

The final report must state whether the two deployments are equivalent, whether both are currently active and what operational risks arise from maintaining both. Make a recommendation, but do not disable either provider.

3.8 Audit deliverables

Produce:

1. `docs/V1_REALITY_AUDIT.md`
2. `docs/V1_SECURITY_FINDINGS.md`
3. `docs/HOSTING_AND_DEPLOYMENT_MAP.md`
4. A feature inventory with the status classifications above.
5. A proposed gap map against this directive and the existing V2

requirements.

1. A list of urgent risks and recommended fixes.
2. A phased implementation proposal with dependencies and acceptance

criteria.

Do not create these files until the repository's existing documentation conventions have been inspected. If equivalent documents already exist, update or consolidate them rather than creating duplicates.

Stop after presenting the audit. Wait for approval before implementation.

---

4. Repository documentation cleanup and governance

The project needs a clear, current source of truth. Documentation should explain the real application without exposing credentials or becoming a second, contradictory specification.

4.1 Superseded handover document

The previous handover document is identified in its contents as `what_i_have_done.md` (the owner has also referred to it as `what_i_did.md`).

During the repository audit:

1. Search for the exact filename and inspect its contents.
2. Confirm whether it is the old V1 handover document described above.
3. Extract any still-useful architectural information into the new,

verified documentation.

1. Correct claims that are inaccurate or no longer current.
2. Ensure the new documentation does not contain PINs, passwords,

service-role keys, access tokens or other secrets.

1. Only after the replacement documentation is complete and checked,

remove the superseded handover Markdown file from the tracked GitHub repository.

1. Record the deletion in the Git change summary.

Do not blindly delete every file containing `what_i_did` or `what_i_have_done` in its name. Confirm the exact file and its purpose first. If a similarly named file is a different useful document, preserve it and ask the owner before removal.

Do not delete the old file before its useful information has been transferred. Do not erase its Git history; a normal tracked-file deletion is sufficient.

4.2 What must not be deleted

Do not delete:

- Source code or components.
- Existing routes/pages.
- Database migration files.
- Supabase configuration.
- Storage assets or academic documents.
- Logo files or other brand assets.
- Environment-variable templates that contain placeholders only.
- Package manifests or lockfiles.
- Build/deployment configuration for Netlify or Vercel.
- Existing useful tests.
- Existing documentation that is still accurate and useful.
- Any file merely because it looks old, duplicated or unfamiliar.

For suspected duplicates, identify references and dependencies first. Propose removal and wait for approval if the impact is unclear.

4.3 Recommended documentation structure

Adapt this to the repository's current structure; do not create duplicates if equivalent files already exist.

- `README.md` --- concise project overview, local setup, scripts and

links to the main docs.

- `docs/PROJECT_OVERVIEW.md` --- product purpose, principles and user

groups.

- `docs/ARCHITECTURE.md` --- verified frontend/backend architecture

and data flows.

- `docs/DATABASE_AND_RLS.md` --- schema, relationships, policies and role boundaries.
- `docs/SECURITY.md` --- security model, upload controls, secrets handling and incident guidance.
- `docs/FEATURE_STATUS.md` --- working, partial, visual-only, broken and planned features.
- `docs/ROADMAP.md` --- approved phases and acceptance criteria.
- `docs/DEPLOYMENT.md` --- Netlify and Vercel configuration map, without secret values.
- `docs/ADMIN_HANDOVER.md` --- executive responsibilities and safe handover instructions.
- `docs/CONTENT_GOVERNANCE.md` --- resource, notice, opportunity and feedback review procedures.
- `docs/CHANGELOG.md` --- meaningful changes by release.

Never place actual secret values in documentation. Use variable names and instructions for setting them securely.

---

5. Approved product direction and implementation roadmap

The following roadmap combines the existing V1 foundation, the association's manifesto commitments and the proposed product improvements. It is not permission to implement every phase immediately. Each phase requires approval after the audit.

Release 1 --- Institutional readiness and manifesto commitments

This is the first product priority after the audit and approval of urgent security corrections.

5.1 Security and governance foundation

Before expanding student-facing functionality:

- Correct critical weaknesses found in admin authentication and authorisation.
- Ensure privileged operations are enforced by trusted backend/database rules.
- Restrict private records to authorised roles.
- Make anonymous uploads safe and controlled.
- Preserve public reads for approved academic resources.
- Establish audit trails for sensitive administrative actions.
- Establish a maintainable executive role and handover process.
- Keep private welfare and feedback records out of public queries and AI retrieval.

Do not make the public resource library require student login as a shortcut to solving security issues.

5.2 Academic Hub improvements

Improve the existing academic library rather than replacing it.

Each resource should support the metadata that is relevant and available, such as:

- Course code and course title.
- Academic level.
- Semester, where known.
- Resource type (past question, lecture note, tutorial, practical/lab material, solution, reference material).
- Examination year/session, where applicable.
- Topic/chapter, when reliably known.
- Source or contributor, where appropriate.
- Date uploaded.
- Review/verification status.
- File type and size.
- Availability/status.

Requirements:

- Search by course code, course title and resource title.
- Filter by level, resource type and year/session where data supports it.
- Make course and level pages useful and avoid empty pages at launch.
- Clearly distinguish verified, unreviewed and rejected material.
- Do not fabricate metadata to make filters look complete.
- Keep one-tap public downloads for approved public resources.
- Preserve mobile-first cards and accessible controls.
- Provide clear empty, loading, unavailable and error states.
- Identify broken links and missing files.
- Keep source attribution and update dates visible where useful.

Topic-level search should be introduced only when topics can be assigned reliably. Do not pretend a document has been indexed by topic merely because its filename contains a keyword.

5.3 Where to Get Help page

Implement a dedicated, easy-to-find page that helps students identify the appropriate support channel.

It should contain verified information such as:

- The type of issue.
- The appropriate office, officer, committee or channel.
- What the student should do.
- Available contact information or official links.
- Relevant opening/availability information, if verified.
- A last-checked or last-updated date.

Possible categories include academic concerns, course/resource problems, departmental enquiries, welfare concerns and technical problems with the platform.

Use only confirmed contact details and procedures. If information is missing, flag it for the association to supply rather than inventing it.

Make the page maintainable by authorised executives and suitable for handover to a new administration.

5.4 Private Feedback Centre

Implement the manifesto's private feedback/reporting commitment as a focused workflow, not an oversized case-management product.

The form should support:

- General student feedback.
- Suggestions and observations.
- Appreciation.
- Incorrect website information.
- Broken links or pages.
- Missing or incorrect academic materials.
- Other relevant student concerns that can safely be handled through this channel.

Suggested fields:

- Category.
- Subject/title.
- Description.
- Optional affected page/resource URL for website reports.
- Optional contact method if the student wants a response.

The product owner must decide the final identified/anonymous submission model. Do not promise anonymity unless the technical design supports it and the privacy notice accurately explains what is collected.

Requirements:

- Do not require an account unless the audit demonstrates a compelling security or workflow reason and the owner approves it.
- Minimise collection of sensitive personal information.
- Clearly tell students what the form is and is not suitable for.
- Restrict reviewer access to authorised Welfare Board members or designated reviewers.
- Keep reviewer-only notes separate from any student-facing response.
- Support a simple status lifecycle: `Received`, `Under Review`, `Referred`, `Resolved`, `Closed`.
- Provide a safe route for technical website reports to reach the responsible administrator.
- Keep an audit trail of status changes and reviewer actions.
- Do not expose submissions through public tables, public storage or unrestricted analytics.
- Do not send private welfare submissions to an AI assistant or include them in public search.
- Provide clear confirmation after submission and explain how follow-up works.

Do not build private messaging, complex case assignment, automated sensitive-risk scoring or AI welfare triage as part of this initial scope.

5.5 Notices and deadlines

Improve the existing notices system.

Each notice should support, where applicable:

- Title and summary.
- Category.
- Intended audience/level.
- Publication date.
- Relevant event/deadline date.
- Official source or link.
- Last-checked date.
- Status (draft, published, expired/archived).
- Attachment/poster where appropriate.
- Author/editor and audit history.

Requirements:

- Clearly distinguish confirmed information from provisional information.
- Display official source links where available.
- Do not imply that a notice is official if it has not been verified.
- Allow authorised executives to draft, review, publish, edit and archive notices.
- Preserve the existing WhatsApp distribution practice. The website should become the searchable source of truth and archive, not an assumed replacement for WhatsApp.
- Avoid automated publication from messaging groups until a safe approval workflow is designed.

5.6 Scholarships and internships board

Improve the existing opportunities page and data model.

Each opportunity should support, where known:

- Opportunity name.
- Provider/organisation.
- Type (scholarship, internship, fellowship, competition, grant, training, etc.).
- Eligibility.
- Academic level or target audience.
- Location/remote status.
- Opening date.
- Deadline.
- Official application URL.
- Source URL.
- Date checked.
- Verification status.
- Publication status.

- Expiry/archive behaviour.

Requirements:

- Show deadlines clearly.
- Mark expired opportunities as expired or archive them automatically

through a reliable scheduled mechanism.

- Do not silently delete historical records.
- Distinguish verified official opportunities from unverified

submissions.

- Check that official links are present and usable before publishing.
- Provide an editorial review process.
- Do not charge organisations for sponsored placement in this release.
- Do not invent deadlines, eligibility or application links.

5.7 Academic calendar

Inspect the current notices and date data before building a separate calendar.

If a dedicated calendar is needed, support:

- Event title and description.
- Start/end date where applicable.
- Audience.
- Official source.
- Confirmed/provisional status.
- Last-checked date.
- Publication and archive status.

Avoid duplicating the same event independently across notices and calendar tables without a clear source-of-truth strategy. A notice may link to a calendar event or share a common underlying record where appropriate.

Release 2 --- Academic engagement and active learning

Build these features after the core library metadata and content governance are dependable.

5.8 Topic-based past-question discovery

Allow students to discover questions by topic/chapter, course, year/session and other reliable metadata.

Start with a manageable workflow:

1. Define a controlled topic taxonomy with academic input.
2. Associate questions or question pages with topics.
3. Allow authorised editors to correct topic assignments.
4. Provide topic filters and related questions.
5. Record the provenance of topic labels.

Do not promise automatic topic classification until its accuracy has been evaluated. If PDF text extraction or OCR is introduced, preserve the original PDF and report extraction failures.

5.9 Verified solutions

Support worked solutions where the association can obtain or review them.

Requirements:

- Clearly distinguish official/lecturer-provided solutions from

student-contributed solutions.

- Record solution author/source and review status.
- Support correction and moderation.
- Do not display unreviewed student answers as authoritative.
- Provide a way to report a suspected error.
- Preserve academic integrity and copyright considerations.

5.10 Interactive quizzes and flashcards

Consider converting suitable academic material into practice activities.

Start with a small pilot for a course with sufficient reviewed content.

Requirements:

- Questions and answers must be reviewed.
- Show explanations or source references where possible.
- Support mobile interaction.
- Track completion only where the student has consented or the feature

requires it.

- Avoid public leaderboards that expose sensitive student performance.
- Make clear that practice scores are not official course grades.

5.11 Student contributions and peer learning

Improve the current anonymous contribution workflow with:

- Submission receipt and status tracking.
- Clear upload rules.
- Moderation feedback.
- Duplicate detection where practical.
- Optional attribution/credit where the contributor consents.
- A reviewed route for student-submitted solutions.

Do not create a public discussion forum until moderators, rules, reporting tools and ownership are defined.

Release 3 --- Advanced academic intelligence

These are advanced features, not launch blockers.

5.12 AI-powered academic search

Potential capabilities:

- Natural-language search across approved academic materials.
- Search by topic, course and concept.
- Similar-question discovery across examination years.
- Answers grounded in approved resources.
- Page-level citations to source PDFs.
- Clear links to open the original source.

Technical requirements before implementation:

- Establish document extraction and OCR quality.

- Preserve page boundaries and source identifiers.
- Design retrieval and indexing around approved public academic materials only.
- Evaluate answers against a representative set of real academic questions.
- Require citations for factual claims derived from documents.
- Show when no reliable source was found.
- Avoid presenting generated solutions as verified official answers.
- Protect against prompt injection or malicious instructions embedded in uploaded documents.
- Do not index private feedback, welfare submissions, personal profiles or administrative secrets.
- Check provider cost, data handling, rate limits and free-tier sustainability before selecting an AI service.
- Do not send sensitive student data to third-party AI services.

Do not implement an AI chatbot merely to add an AI label to the website. Its answers must be demonstrably useful and source-grounded.

5.13 Similar-question discovery

Explore methods such as text similarity, embeddings or topic matching only after document extraction and metadata quality are adequate.

The system should:

- Show the original question and source year/session.
- Explain why a question is considered similar where practical.
- Avoid implying that similar questions will appear in future examinations.
- Allow academic reviewers to report poor matches.

5.14 Personalised student dashboard

Preserve the public, no-login resource experience.

If optional student accounts are introduced later, they may support:

- Cross-device bookmarks.
- Saved resources.
- Preferences such as academic level and courses.
- Relevant upcoming deadlines.
- Recent activity.
- Optional study progress.

Requirements:

- Account creation must remain optional for public resource consumption.
- Explain what information is stored and why.
- Allow students to control or remove personal preferences where appropriate.

- Avoid inferring academic ability or sensitive traits from browsing

behaviour.

- Do not claim to track academic performance without reliable data.

5.15 Meaningful engagement and recognition

If points, badges or streaks are introduced, reward useful activity such as:

- Submitting a resource that passes review.
- Correcting an inaccurate listing.
- Contributing a reviewed solution.
- Completing an academic practice activity.
- Helping improve the quality of the resource library.

Do not reward empty logins, repeated clicks or spam submissions. Do not make public ranking of students' academic performance a default.

---

6. Features to postpone or exclude from the current scope

Do not implement these unless the owner explicitly revisits the decision:

- Paid subscriptions for core academic resources.
- Premium paywalls around past questions or lecture notes.
- Paid sponsored scholarship placements.
- A textbook buy/sell marketplace.
- Private direct messaging between students.
- A large general-purpose social network.
- A general discussion forum without a moderation plan.
- A Pomodoro timer as a standalone priority feature.
- Multiple-language infrastructure before there is a demonstrated

need.

- Automated publication of unreviewed WhatsApp/Telegram messages.
- AI access to private welfare or feedback records.
- AI-generated academic answers presented as verified solutions.
- Unnecessary collection of student personal data.

These are not necessarily bad ideas forever. They are outside the current implementation priority because they add operational, privacy, moderation or financial responsibilities without being essential to the immediate product mission.

---

7. Data, privacy and security requirements

These requirements apply to every implementation phase.

7.1 Secrets and environment variables

- Never commit `.env` files containing real credentials.
- Never expose Supabase service-role keys, database passwords, private

API keys, access tokens or signing secrets in frontend code.

- Public/publishable client keys may be used only as intended by

Supabase and must be protected by correct RLS and storage policies.

- Use the hosting providers' environment-variable facilities for deployment secrets.
- Maintain a safe `.env.example` containing variable names and placeholders only.
- Do not print secret values in logs, audit reports or documentation.
- If a secret is found in Git history or a tracked file, report it and recommend rotation. Removing the current file alone does not invalidate a leaked credential.

7.2 Database access

- Enable and correctly configure RLS on tables containing user or administrative data.
- Apply least privilege.
- Ensure public users can read only intended public content.
- Ensure private submissions and profiles are not publicly readable.
- Restrict writes to authorised roles and validated workflows.
- Validate data on trusted server/database boundaries, not only in the browser.
- Protect administrative changes with audit logging.
- Avoid broad policies such as unrestricted public INSERT/UPDATE/DELETE unless a documented threat assessment justifies them.

7.3 File uploads

- Validate file type, extension, size and content where feasible.
- Use safe object naming and prevent accidental overwrite.
- Keep unreviewed submissions separate from publicly published resources.
- Limit upload volume and abuse.
- Do not allow uploaded files to execute as application code.
- Keep public download permissions separate from upload permissions.
- Provide moderation and rejection paths.
- Consider malware scanning or a safe quarantine process if the infrastructure supports it within budget.

7.4 Privacy

- Collect only what is necessary.
- Explain the purpose of collected data.
- Do not expose personal data in public analytics.
- Do not expose private feedback or welfare records to general administrators unless their role requires it.
- Define retention and deletion practices for sensitive submissions.
- Do not promise anonymity unless the architecture supports the promise.

- Do not use private student records for AI training, indexing or retrieval.

7.5 Accessibility and reliability

- Use semantic HTML and accessible labels.
- Support keyboard navigation and visible focus.
- Maintain readable contrast and text sizing.
- Provide loading, empty, success and error states.
- Avoid controls that look interactive but do nothing.
- Preserve mobile performance and low-bandwidth usability.
- Avoid unnecessary large dependencies.
- Ensure important information is not conveyed by colour alone.

---

8. Technical implementation rules

8.1 Follow existing conventions

Before changing code:

- Inspect relevant components and their callers.
- Follow existing naming, folder and styling conventions.
- Reuse existing components where appropriate.
- Avoid introducing dependencies for functionality already supported by the project.
- Explain why a new dependency is necessary.
- Keep changes focused and reviewable.

8.2 Database migrations

- Use version-controlled migrations for schema and policy changes.
- Never make undocumented manual production schema changes.
- Review migrations before applying them.
- Make migrations safe and repeatable where practical.
- Preserve existing records.
- Include rollback or recovery guidance for risky changes.
- Do not reset or reseed production data.
- Do not drop or rename existing columns/tables without an approved migration plan.

8.3 Git workflow

- Inspect `git status` before starting.
- Do not overwrite uncommitted user work.
- Create a clear checkpoint before a substantial approved phase.
- Use focused commits with descriptive messages.
- Do not force-push or rewrite shared Git history.
- Do not commit build artefacts, local databases, secrets or temporary

files.

- Summarise every changed, added and deleted file.

8.4 Testing

For each phase:

- Run the existing lint/type-check/test commands where available.
- Run the production build.
- Test affected routes on mobile and desktop.
- Test loading, empty and failure states.
- Test anonymous access and authorised access separately.
- Test database policies for the relevant roles.
- Test upload and download workflows where affected.
- Test both Netlify and Vercel preview/deployment behaviour when configuration changes affect them.
- Do not claim a test passed unless it was actually run.

8.5 Free-tier and cost discipline

Before adding an external service:

- Identify the provider and exact feature required.
- Check current free-tier limits and usage constraints.
- Estimate expected usage.
- Identify what happens when quotas are reached.
- Provide a graceful fallback.
- Avoid background jobs or polling that can exhaust quotas.
- Obtain approval before introducing paid services or recurring costs.

---

9. Release acceptance criteria

A phase is not complete because its interface has been generated.

9.1 Institutional readiness acceptance

- Existing public resource browsing and downloads still work.
- No compulsory login has been introduced for public academic resources.
- Admin actions are protected by backend-enforced authorisation.
- Private feedback records cannot be read by anonymous users or unauthorised roles.
- Anonymous uploads cannot bypass moderation and validation controls.
- Where to Get Help contains verified information.
- Every level has useful academic resources or an explicitly documented content gap.
- Notices show reliable sources and relevant dates where available.
- Opportunities show deadlines, official links and expiry status.

- Mobile layouts and core workflows work.
- Both hosting deployments remain intact unless a separately approved change says otherwise.
- Documentation and handover instructions reflect the actual implementation.

9.2 Academic engagement acceptance

- Topic filters return relevant, correctly labelled resources.
- Solutions show their review status and source.
- Practice questions have been academically checked.
- Student contributions enter a controlled review process.
- Recognition mechanisms reward meaningful activity and resist obvious abuse.
- No private student data is exposed through public leaderboards or analytics.

9.3 AI academic intelligence acceptance

- Answers are grounded in approved source documents.
- Citations identify the correct source and page.
- The source document can be opened from the answer.
- Unsupported answers are clearly identified as unsupported.
- Similar-question results are evaluated for relevance.
- Private welfare, feedback and administrative records are excluded.
- Provider cost, rate limits, privacy and failure behaviour are documented.
- The feature has been tested against representative academic questions.

---

10. Required reporting format after every phase

At the end of each approved phase, provide a concise but complete engineering report with:

1. **Phase completed:** What was implemented.
2. **Files changed:** Added, modified and deleted files.
3. **Database changes:** Migrations, policies, functions, triggers and affected tables.
1. **Security implications:** What changed and what remains unresolved.
2. **Tests performed:** Exact commands and actual results.
3. **User workflows tested:** Include mobile and desktop where relevant.
1. **Hosting implications:** Effects on Netlify and Vercel, if any.
2. **Outstanding issues:** Bugs, missing data, limitations and risks.
3. **Rollback/recovery:** How to reverse or recover from risky changes.

4 **Next recommended step:** One proposed next phase, with dependencies.

Do not report "complete" when a feature is only visually implemented or has not been tested end-to-end.

---

11. First instruction to the AI receiving this document

Your immediate assignment is:

Perform the read-only V1 Reality Audit described in Section 3.

Do not begin Release 1 implementation yet.

Specifically:

1 Inspect the existing GitHub repository and current branch.

2 Inspect the existing application architecture and all routes.

3 Verify the reported Supabase schema, RLS policies, storage permissions and data flows using authorised read-only access.

1 Audit the existing admin gate and determine whether privileged actions are protected on the backend.

1 Inspect resource submissions, public downloads, Student Locker behaviour and analytics collection.

1 Inspect notices, opportunities and academic content metadata.

2 Inspect both the Netlify and Vercel deployments and document their relationship.

1 Identify the old V1 handover document by its actual contents; plan its safe replacement and removal from the tracked repository as described in Section 4.

1 Compare the actual implementation against the roadmap in this directive.

1 Produce the audit deliverables and a prioritised implementation proposal.

Do not edit code, run destructive commands, apply migrations, change Supabase policies, modify production data, change environment variables, delete files, change domains, disconnect hosting providers or deploy anything during this first assignment.

If you need to run a command that could modify the repository or external services, explain it and request approval first.

Stop after presenting the audit and wait for the project owner to approve the next step.

---

12. Final instruction: protect the institution, not just the code

ASSON UI belongs to the association and should outlive the current executives and the person who initially built it.

The objective is not to maximise the number of features or produce the most complicated architecture. The objective is to build a dependable, secure, useful and maintainable academic platform that students trust and future administrations can operate.

Preserve what works. Verify what is claimed. Fix what is unsafe. Implement the manifesto commitments. Add advanced capabilities only when the content, governance and technical foundations can support them.

Continue the existing ASSON UI project. Do not rebuild it.